PINITHUB / INTERNSHIP ENGINEERING PROGRAMME

AI Image Edit Detection & Forensic Analysis System

Step-by-step technical development plan for an intern

SCOPE	Detect and localize image edits: retouching, object insertion and removal, compositing, generative fill, AI enhancement, upscaling, filters, cropping and partial replacement.
STACK	React + TypeScript (Vercel) | Node.js + Express + TypeScript + Prisma (Render) Python + FastAPI + PyTorch forensics service | Supabase PostgreSQL + Supabase Storage
FOR	Shashikumar Mulimani, internship candidate
DEADLINE	Complete within one week. 10 phases, each with learn / build / test / deliver and acceptance criteria
GOAL	Detect every kind of edit. Every result is measured, so the system keeps getting better.

Prepared by the PinITHUB Team. A standalone project: it uses no PinITHUB source code, data or internal design.

Version 1.1 | Prepared for Shashikumar Mulimani | To be completed within one week

CONTENTS

Table of contents

1. How to use this plan . . . 4

1.1 Ground rules . . . 4

1.2 What "done" means for the whole project . . . 4

2. The problem and what is realistically detectable . . . 5

3. System architecture . . . 6

3.1 Components and responsibilities . . . 6

3.2 End-to-end data flow . . . 7

4. Technology stack and why each piece was chosen . . . 7

5. Detection methods: what each one sees . . . 8

5.1 Metadata and provenance . . . 8

5.2 Compression forensics (JPEG) . . . 8

5.3 Noise and sensor traces . . . 9

5.4 Geometry: copy-move and resampling . . . 9

5.5 Learned localisation models (general manipulation) . . . 9

5.6 Generative and AI-specific detectors . . . 9

5.7 Reference comparison (when the original is provided) . . . 10

5.8 Fusion, calibration and the final decision . . . 10

6. Datasets . . . 10

6.1 Public datasets **[CHECK LICENCE]** . . . 10

6.2 Your own generated dataset . . . 11

7. Database and storage design (Supabase) . . . 12

7.1 Tables . . . 12

7.2 Prisma schema excerpt . . . 12

7.3 Claiming a job safely . . . 13

7.4 Security and storage rules . . . 13

8. Evaluation: how coverage and accuracy are measured . . . 14

8.1 Image-level metrics (is this image edited?) . . . 14

8.2 Pixel-level metrics (where?) . . . 14

8.3 Calibration . . . 14

8.4 Generalisation and robustness . . . 14

8.5 Report template . . . 14

9. Repository, setup and configuration . . . 15

9.1 Folder structure . . . 15

9.2 Prerequisites . . . 15

9.3 Setup steps . . . 15

9.4 Environment variables . . . 16

10. API design . . . 16
10.1 Result shape . . . 17
10.2 API rules . . . 17
11. Implementation phases . . . 17
Phase 0: Foundations . . . 17
Phase 1: Dataset and ground truth . . . 18
Phase 2: Platform skeleton (end to end with a dummy detector) . . . 18
Phase 3: Classical detectors . . . 18
Phase 4: Reference comparison mode . . . 19
Phase 5: Pretrained deep models . . . 19
Phase 6: Fine-tuning and generative edits . . . 19
Phase 7: Fusion, calibration, false-positive control . . . 20
Phase 8: Full evaluation and unseen tools . . . 20
Phase 9: UI polish, deployment, handover . . . 20
12. Testing strategy . . . 21
12.1 Inputs every test suite must include . . . 21
13. Deployment . . . 21
14. Risks and limitations . . . 22
14.1 Proven versus experimental at a glance . . . 22
15. Technical references . . . 22
16. Final acceptance checklist . . . 23

READ THIS FIRST

1. How to use this plan

This document tells you, step by step, how to build an **image-edit detection and forensic analysis system** from an empty repository to a deployed, measured product, **within one week**. It is written so that you never have to guess a missing step: each phase says what to **learn**, what to **build**, how to **test** it, what to **deliver**, and the **acceptance criteria** a reviewer will check before you move on.

Prepared for Shashikumar Mulimani: deadline one week
Complete this plan **within one week** of receiving it. Work through the phases in order (Section 11): each phase builds on the one before. At the end of the week, deliver everything that is finished and tested, with the evaluation numbers you measured, and a short note on anything not finished and what it still needs.

1.1 Ground rules

- **Standalone project.** You build everything from scratch. You do not get access to any PinITHUB repository, database, keys or internal design, and you must not try to. If later integrated, it will connect through the API contract in Section 10.
- **Aim for 100%, prove every number.** Push to catch every kind of edit, and back every claim with a test: name the dataset, the edit type, the metric and the number you measured, then show what was caught and what to improve next.
- **Label every technique honestly.** This plan tags techniques as [PROVEN] (well established, works in practice within known limits), [PROVEN, LIMITED] (works, but only in specific conditions) or [EXPERIMENTAL] (active research; results vary and generalise poorly). Keep these tags in your own documentation.
- **Licences matter.** Many forensic models and datasets are released for non-commercial research only. Anything tagged [CHECK LICENCE] must be checked and recorded in `docs/LICENCES.md` before you use it.
- **Ask early.** If a step is unclear, write down what you tried and ask your mentor straight away.

1.2 What "done" means for the whole project

Area	Done when
Product	A user uploads an image (and optionally the original) and gets: edited / not edited / inconclusive, a calibrated probability, pixel masks for suspected regions, a heatmap and the evidence behind it.
Measurement	A written evaluation report with image-level and pixel-level metrics per edit type, per edit size and per editing tool, including tools never seen in training, and false-positive rates on unedited but normally processed photos.
Engineering	Deployed web app, API and worker; database migrations; tests in CI; documented setup that a new person can follow in under one hour.
Honesty	A limitations section that names the edits the system misses and how often.

PROBLEM DEFINITION

2. The problem and what is realistically detectable

Image editing leaves traces, but different edits leave different traces, and some leave almost none. A good system combines several independent detectors and is honest about which edits it can and cannot see. The table below is the most important table in this plan: it sets expectations before you write any code.

Edit type	Typical trace	Blind detection (no original)	With the original
Splicing / compositing (paste from another photo)	Noise, compression and lighting inconsistencies at the pasted region	[PROVEN] good on JPEG splices, weaker after heavy recompression	[PROVEN] near-certain
Copy-move (clone inside the same photo)	Duplicated keypoints and patches	[PROVEN] strong unless the clone is tiny or heavily blended	[PROVEN]
Object removal (classic healing / content-aware fill)	Smoothed noise, repeated texture	[PROVEN, LIMITED] depends on area and texture	[PROVEN]
Generative fill / AI inpainting (diffusion, LaMa)	Decoder fingerprints, frequency and noise mismatch	[EXPERIMENTAL] works on tools seen in training, drops on unseen tools	[PROVEN] strong
Fully AI-generated image	Generator frequency patterns	[EXPERIMENTAL] detectors exist but generalise poorly	n/a
Retouching (skin smoothing, liquify)	Local smoothing, warped textures	[PROVEN, LIMITED]	[PROVEN]
Tiny edits (under 1% of the area, a few edge pixels)	Very weak, often below noise	[PROVEN, LIMITED] frequently missed: measure and report	[PROVEN] if aligned to the pixel
Resizing / upscaling (classic)	Periodic interpolation correlations	[PROVEN] for clean resampling	[PROVEN]
AI upscaling / enhancement	Hallucinated texture, decoder patterns	[EXPERIMENTAL]	[PROVEN]
Global filters / colour grading	Histogram and colour statistics shift	[PROVEN, LIMITED] hard to tell from camera processing	[PROVEN]
Cropping	JPEG 8x8 grid misalignment, missing EXIF consistency	[PROVEN, LIMITED] only for JPEG crops off the 8-pixel grid	[PROVEN] via registration
Partial image replacement	Mixed compression histories	[PROVEN, LIMITED]	[PROVEN]

Why two modes

The system must support **blind analysis** (only the suspect image) and **reference analysis** (the suspect image plus the original). Reference analysis is far more reliable because you can align the two images and compare them directly. Many real uses (a creator checking a copy of their own photo) have the original, so build reference mode early (Phase 4).

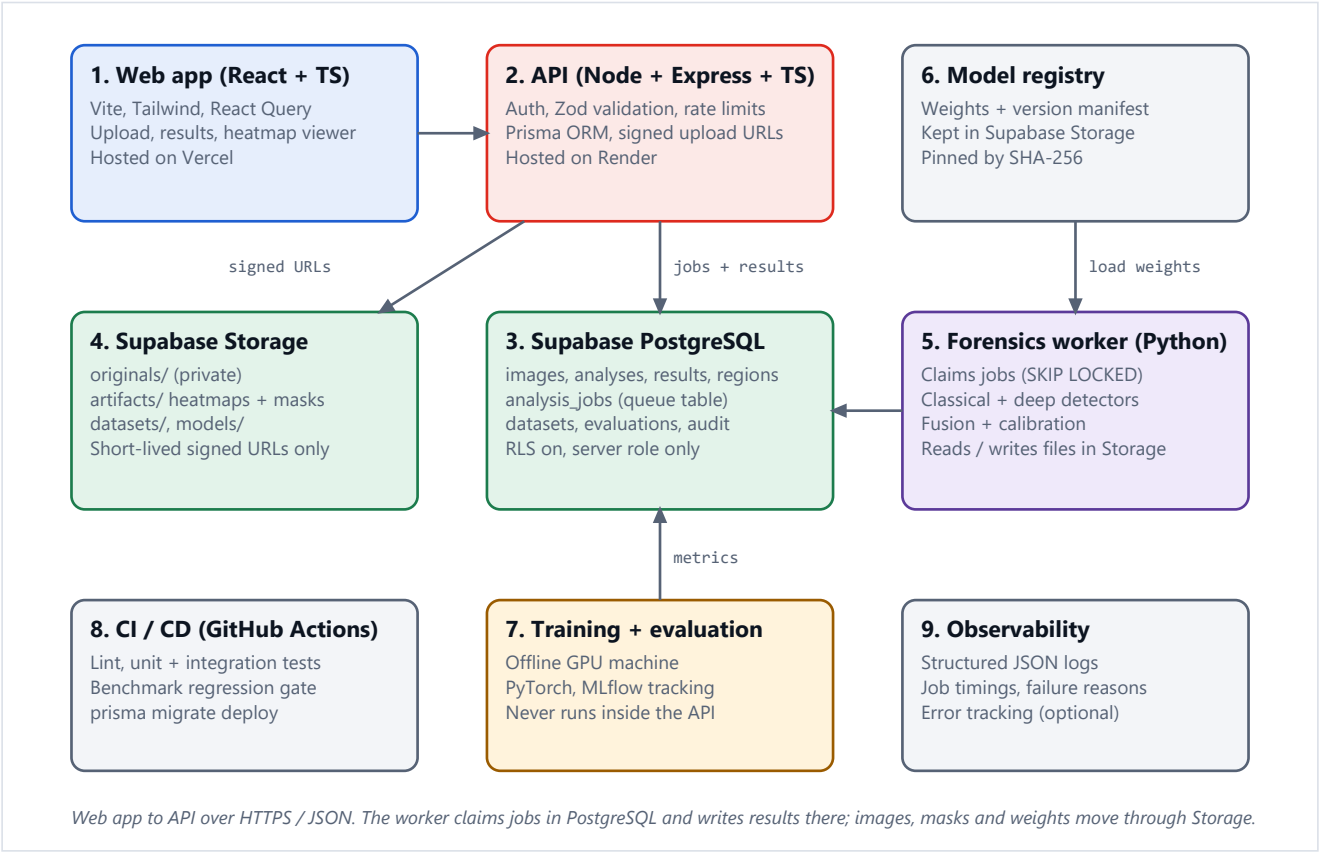
What the system must never say

Never output "authentic", "unedited for certain" or "100%". Say "no evidence of editing found by the current detectors". A clean result is the absence of detected evidence, not proof of authenticity.

ARCHITECTURE

3. System architecture

The system is four deployable parts (web app, API, forensics worker, database and storage) plus an offline training and evaluation environment. The API never runs heavy image analysis itself: it records a job, and a separate Python worker does the work. That keeps the API fast and lets the worker scale or move to a GPU without changing anything else.



Web app to API over HTTPS / JSON. The worker claims jobs in PostgreSQL and writes results there; images, masks and weights move through Storage.

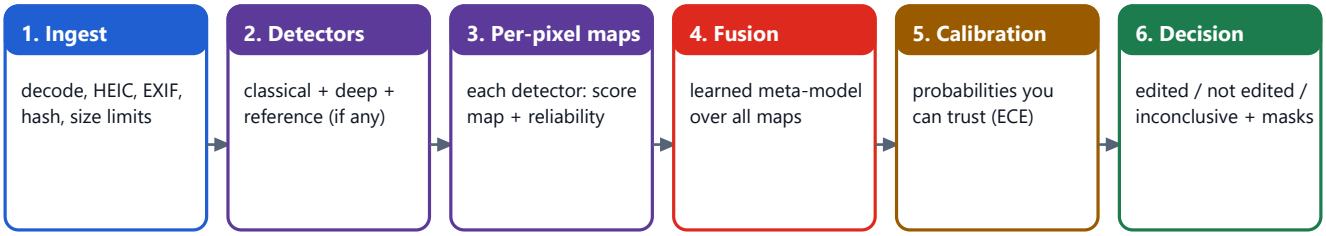
3.1 Components and responsibilities

#	Component	Responsibility	Must never
1	Web app (React + TypeScript)	Upload (with progress), optional original upload, polling a job, showing verdict, masks, heatmap overlay with an opacity slider, evidence list, PDF export of a report.	Talk to the database or storage with privileged keys.
2	API (Node + Express + TypeScript)	Auth, input validation, size and type checks, creating signed upload URLs, inserting jobs, serving results, rate limits, audit log.	Decode or analyse images in the request.
3	PostgreSQL (Supabase)	Source of truth: images, jobs, results, regions, detector outputs, model versions, datasets, evaluation runs.	Hold image bytes (only paths and hashes).
4	Storage (Supabase)	Original images, uploaded references, heatmaps, masks, dataset images, model weights.	Be public. All buckets private.
5	Forensics worker (Python)	Claims jobs, runs detectors, fuses and calibrates, writes masks, heatmaps and results.	Accept work from the internet directly.
6	Model registry	Versioned model weights with SHA-256 checksums and a manifest (name, version, licence, training data).	Load a model whose checksum does not match.
7	Training + evaluation	Dataset generation, training, fine-tuning, benchmarks, metric reports.	Run inside the production worker.

#	Component	Responsibility	Must never
8	CI / CD	Tests, lint, type checks, benchmark regression gate, migrations on deploy.	Deploy when tests fail.

3.2 End-to-end data flow

1. The web app asks the API for a **signed upload URL** (`POST /v1/uploads`), then uploads the image directly to the private `originals` bucket. Large files never pass through the API server.
2. The web app calls `POST /v1/analyses` with the upload id (and an optional reference upload id). The API checks ownership, size, type and quota, writes an `analyses` row and an `analysis_jobs` row with status queued, and returns 202 with the analysis id.
3. A worker claims the oldest queued job with `SELECT ... FOR UPDATE SKIP LOCKED`, sets it to `running`, downloads the image, decodes it (including HEIC), records its SHA-256 and EXIF.
4. The worker runs every enabled detector. Each one produces an image-level score, a per-pixel score map and, where available, a reliability map. Each output is written to `detector_outputs` and its map to storage.
5. The fusion model combines the maps into one probability map; calibration turns scores into trustworthy probabilities.
6. The decision step thresholds the map into pixel masks, splits them into regions, filters tiny noise, and decides edited / not edited / inconclusive. Masks, heatmap and the final result are written; the job becomes done.
7. The web app polls `GET /v1/analyses/:id` (or listens via Supabase Realtime) and shows the result with short-lived signed URLs for the heatmap and masks.



Every stage writes its output to the database, so any verdict can be traced back to the evidence that produced it.

TECHNOLOGY

4. Technology stack and why each piece was chosen

The stack mirrors a common, well-supported production setup: TypeScript for the product, Python for the science, PostgreSQL for the data. Pin exact versions in lock files; the versions below are the minimums to target.

Layer	Technology	Why
Web app	React 18, TypeScript 5, Vite 5, Tailwind CSS 3, React Query (TanStack), React Router	Fast builds, typed UI, simple data fetching with caching and polling.
Web hosting	Vercel	Zero-config static hosting with previews per pull request.
API	Node.js 20 LTS, Express 4, TypeScript 5, Zod (validation), Pino (logs), Helmet, express-rate-limit	Mature, typed, easy to test; Zod gives one source of truth for request shapes.
ORM + migrations	Prisma 5	Typed queries and versioned SQL migrations. Works with Supabase PostgreSQL (see 9.4 for pooler settings).
Database	Supabase PostgreSQL 15+	Managed Postgres with backups, row-level security, Realtime and a connection pooler.

Layer	Technology	Why
File storage	Supabase Storage (S3-compatible)	Private buckets, signed URLs, same project and auth as the database.
Forensics service	Python 3.11, FastAPI, Uvicorn, Pydantic 2	The forensics and ML ecosystem is in Python; FastAPI gives typed health and admin endpoints.
Image I/O	Pillow + pillow-heif, OpenCV 4, jpegio, exiftool / piexif	HEIC phone photos, fast decoding, direct access to JPEG DCT coefficients and quantisation tables, full metadata.
Numerics	NumPy, SciPy, scikit-image	Filters, statistics, connected components, SSIM.
Deep learning	PyTorch 2, torchvision, timm, segmentation-models-pytorch, albumentations	Standard for research models; easy fine-tuning of segmentation backbones; strong augmentation.
Generative data	diffusers (Stable Diffusion inpainting), LaMa	Create AI-inpainted training and test images with exact masks.
ML ops	MLflow (tracking), DVC or a manifest table (dataset versions)	Every result traceable to code, data and model versions.
Provenance	C2PA content credentials (c2pa-python / c2patool)	Read signed edit history when a file carries it.
Testing	pytest, Jest, Supertest, Playwright (UI smoke)	Unit, API and browser tests in one CI pipeline.
Infra	Docker, GitHub Actions, Render (API + CPU worker), a rented GPU for training and optional GPU inference	Reproducible builds; GPU only where it pays off.

Keep the server and the database close
Choose the Supabase region and the Render region to be the same (or as close as possible). Every database round trip crosses that distance; an API 200 ms away from its database spends seconds per request on a page that runs ten queries.

FORENSIC ALGORITHMS AND MODELS

5. Detection methods: what each one sees

No single detector covers the edit types in Section 2. The system runs a **set of independent detectors**, each strong on some edits and blind to others, and a fusion model learns how to combine them. Implement them in the order of the phases.

5.1 Metadata and provenance

- [PROVEN] **EXIF / XMP / IPTC analysis**: software tags (e.g. an editor name), modification dates later than capture dates, thumbnail that differs from the main image, missing camera fields. Cheap and useful, but metadata is easily stripped or forged, so it can raise suspicion but never clear an image.
- [PROVEN] **C2PA content credentials**: when present, a cryptographically signed record of edits. Verify the signature; report the recorded actions. Absence means nothing.
- [PROVEN] **Quantisation table fingerprint**: JPEG tables identify camera firmware versus editing software.

5.2 Compression forensics (JPEG)

- [PROVEN] **Double JPEG compression** (aligned and non-aligned): regions pasted from another JPEG show a different DCT coefficient histogram. Read coefficients directly with jpegio. Reference: Bianchi and Piva (2012).
- [PROVEN, LIMITED] **Error Level Analysis (ELA)**: recompress and look at the difference. Easy to show, easy to misread; use only as one weak signal and never on PNG.
- [PROVEN, LIMITED] **JPEG grid alignment**: the 8x8 block grid shifted from the origin suggests cropping. Only works for JPEG and crops not on multiples of 8.

- [PROVEN][CHECK LICENCE] **CAT-Net** (Kwon et al., 2021/2022): a network trained on DCT streams; strong for JPEG splicing, weak on PNG and on images recompressed many times.

5.3 Noise and sensor traces

- [PROVEN] **Noise residual inconsistency**: high-pass or denoiser residual, local variance and its statistics; pasted or generated regions often have different noise. Classical baseline: Splicebuster (Cozzolino et al., 2015).
- [PROVEN][CHECK LICENCE] **Noiseprint** (Cozzolino and Verdoliva, 2020): learned camera-model noise pattern; edits break it locally.
- [PROVEN, LIMITED] **PRNU** (Lukas, Fridrich and Goljan, 2006): sensor fingerprint; extremely strong but needs reference images from the same camera, so it is only for special cases.

5.4 Geometry: copy-move and resampling

- [PROVEN] **Copy-move detection**: SIFT or ORB keypoints matched within the same image (g2NN test), clustered, verified with RANSAC; dense block matching (Zernike or PatchMatch) for flat regions. Reference: Amerini et al. (2011).
- [PROVEN] **Resampling detection**: interpolation leaves periodic correlations visible in the spectrum of a linear predictor residual (Popescu and Farid, 2005; Kirchner, 2008). Localises resized or rotated pasted regions.

5.5 Learned localisation models (general manipulation)

These output a per-pixel map. Use pretrained weights first (Phase 5), then fine-tune (Phase 6).

Model	What it is good at	Notes
TruFor (Guillaro et al., CVPR 2023)	Detection + localisation + a reliability map; strong across many manipulations	[PROVEN][CHECK LICENCE] research licence; reliability map is very useful for fusion
IML-ViT (Ma et al., 2023)	Vision-transformer localisation with edge supervision	[PROVEN, LIMITED][CHECK LICENCE]
MVSS-Net (Chen et al., ICCV 2021)	Multi-view (noise + edge) localisation	[PROVEN, LIMITED][CHECK LICENCE]
ManTra-Net (Wu et al., CVPR 2019)	Older general baseline	[PROVEN, LIMITED] use as a baseline only
Your fine-tuned segmentation model	Your own data, including generative inpainting	SegFormer or UNet backbone via segmentation-models-pytorch; you own it

5.6 Generative and AI-specific detectors

> **Treat this section as research**
> Detectors for AI-generated and AI-inpainted content work well on the generators they were trained on and often fail on new ones. You must test them on tools held out from training (Section 8.4) and report the drop honestly.

- [EXPERIMENTAL] **UnivFD** (Ojha et al., CVPR 2023): CLIP features plus a linear probe; better generalisation than older detectors, image-level only.
- [EXPERIMENTAL] **NPR** (Tan et al., CVPR 2024): neighbouring-pixel relationships left by up-sampling layers in generators.
- [EXPERIMENTAL] **DIRE** (Wang et al., ICCV 2023): reconstruction error through a diffusion model; expensive.
- [EXPERIMENTAL] **Inpainting localisation by fine-tuning**: train your segmentation model on images you inpaint yourself with several tools (Phase 6). This is the main lever for generative fill.

- [EXPERIMENTAL] **AI upscaling / enhancement**: frequency-domain features plus a small classifier trained on upscaler outputs (Real-ESRGAN and similar). Report as experimental.

5.7 Reference comparison (when the original is provided)

- [PROVEN] **Registration**: SIFT keypoints, ratio test, RANSAC homography or affine fit (OpenCV) to undo crop, scale and rotation. Record inlier count and reprojection error; refuse to compare if alignment is poor.
- [PROVEN] **Photometric normalisation**: match global colour and exposure so a filter does not light up the whole image; report the global change separately as "filter / colour grading".
- [PROVEN] **Difference maps**: absolute difference, SSIM map and a perceptual distance (LPIPS) on aligned tiles; threshold relative to the noise floor estimated from the image itself.
- [PROVEN] **Crop report**: the registered footprint of the suspect image inside the original gives the exact crop box.

5.8 Fusion, calibration and the final decision

- **Fusion**: a small model (logistic regression first, then a gradient-boosted model or a tiny CNN over stacked maps) takes every detector map and reliability map and outputs one probability map. Train it on the validation split only.
- **Calibration**: temperature scaling or isotonic regression so that "0.8" means right about 80% of the time. Measure with expected calibration error (ECE) and reliability diagrams.
- **Three-way decision**: edited (probability above the high threshold), not edited (below the low threshold), inconclusive (in between, or the image is too small, too compressed or the detectors disagree strongly).
- **Regions**: threshold the map, morphological clean-up, connected components, drop regions below a minimum area, keep each region with its own confidence and a guessed edit type.

DATA

6. Datasets

Model quality is decided by data. You will combine public benchmarks (for comparison with published work) with a dataset you generate yourself (for the edit types and tools that matter here, with exact masks).

6.1 Public datasets [CHECK LICENCE]

Dataset	Contents	Use for
CASIA v2 (Dong et al., 2013)	Splicing and copy-move, image-level labels; masks available from community releases	Baseline training, image-level
Columbia uncompressed (Hsu and Chang, 2006)	Splicing on uncompressed images with masks	Testing noise-based detectors
COVERAGE (Wen et al., 2016)	Copy-move with similar-but-genuine objects	Copy-move false-positive testing
IMD2020 (Novozamsky et al., 2020)	Real-life manipulations with masks	Realistic testing
DEFACTO (Mahfoudi et al., 2019)	Large synthetic splicing, removal, copy-move on COCO	Training at scale
NIST MFC (2019)	Large evaluation set with masks (registration required)	Independent evaluation
CocoGlide (from the TruFor paper)	Diffusion inpainting with masks	Generative fill testing
TGIF (Mareen et al., 2024)	Text-guided inpainting forgeries from several generators	Generative fill, unseen-tool tests

Dataset	Contents	Use for
GenImage (Zhu et al., 2023)	Fully generated images from many generators	AI-generated image detection
RAISE (Dang-Nguyen et al., 2015), Dresden	Unedited camera originals	Negatives: false-positive measurement

6.2 Your own generated dataset

Write a reproducible generator (`services/forensics/datasets/generate.py`) that takes unedited originals and produces edited copies with **exact pixel masks**. Every output is a row in `dataset_items` with its parameters, so any image can be regenerated.

Edit family	How to generate	Mask
Generative fill	Stable Diffusion inpainting (diffusers), at least 2 model versions; LaMa; plus a hand-made set from a commercial tool (e.g. Photoshop Generative Fill), kept for testing only	The inpainting mask
Splicing	Paste objects from another image (COCO segmentation), with and without feathering and colour matching	Object mask
Copy-move	Clone a patch inside the image, with rotation, scale and blending variants	Target region
Object removal	OpenCV inpainting (Telea, Navier-Stokes) and LaMa	Removed region
Retouch	Local smoothing, liquify-style warps (thin-plate spline), local sharpening	Edited region
Tiny edits	Edge changes of 1-50 pixels: recolour a thin strip, move an edge, alter a few characters of text	Exact pixels
Upscale / enhance	Bicubic and AI upscalers (Real-ESRGAN), then crop back to size	Whole image (global)
Filters / grading	Curves, saturation, LUTs, vignette, social-media-style filters	Whole image (global)
Crop	Crops on and off the JPEG grid, with and without resize	Crop box

Edit sizes and post-processing

- **Area buckets**: under 0.5%, 0.5-1%, 1-2%, 2-5%, 5-10%, 10-25%, over 25% of the image. Small edits are the hard, important cases; generate more of them.
- **Post-processing** applied after editing, as real images are shared: JPEG at quality 95, 85, 75 and 60; resize to 50% and 75%; a WhatsApp-like and an Instagram-like pipeline (resize to a long side of 1600 or 1080, recompress); PNG with no recompression.
- **Negatives are as important as positives**: unedited originals through the same post-processing. Without them you cannot measure false positives.

Splits and leakage

- Split by **source photo**, never by edited file: all edits of one original go to the same split (train 70%, validation 15%, test 15%). Otherwise the model memorises the background and the test score is fake.
- Hold out whole **tools** for the unseen-tool test (Section 8.4): at least one generative tool must never appear in training.
- Freeze the test set early. Never tune anything on it.

Target sizes (adjust with your mentor)

Set	Minimum
Unedited originals (diverse cameras, phones, scenes, lighting)	2,000
Generated edits across all families and sizes	12,000

Set	Minimum
Hand-made edits from commercial tools (test only)	200
Negatives after post-processing variants	6,000

DATA MODEL

7. Database and storage design (Supabase)

All structured data lives in Supabase PostgreSQL, managed with Prisma migrations. Image bytes live in Supabase Storage; the database stores only the path, size and SHA-256 of each file.

7.1 Tables

Table	Purpose	Key columns
users	Accounts (or use Supabase Auth and keep a profile table)	id (uuid), email, role, created_at
images	Every uploaded file	id, owner_id, storage_path, sha256, mime, width, height, bytes, exif (jsonb), created_at
analyses	One request to analyse an image	id, owner_id, image_id, reference_image_id (nullable), mode (blind | reference), status, verdict, probability, created_at, finished_at
analysis_jobs	Work queue for workers	id, analysis_id, status (queued | running | done | failed), attempts, locked_by, locked_at, error, priority
detector_outputs	One row per detector per analysis	id, analysis_id, detector, detector_version, score, reliability, map_path, ms, details (jsonb)
regions	Suspected edited regions	id, analysis_id, mask_path, bbox (jsonb), area_pct, confidence, edit_type_guess
model_versions	Model registry	id, name, version, sha256, storage_path, licence, training_run_id, active
datasets / dataset_items	Benchmark and generated data	dataset: id, name, version, split_seed; item: id, dataset_id, image_id, mask_path, edit_family, area_pct, tool, post_process, split
evaluation_runs / evaluation_metrics	Reproducible results	run: id, git_sha, model_version_ids, dataset_id, created_at; metric: run_id, slice, metric, value, ci_low, ci_high
audit_events	Who did what, when	id, actor_id, action, target, ip, created_at

7.2 Prisma schema excerpt

```
prisma/schema.prisma (excerpt)

model Analysis {
  id               String           @id @default(uuid()) @db.Uuid
  ownerId          String           @db.Uuid
  imageId          String           @db.Uuid
  referenceImageId String?          @db.Uuid
  mode             AnalysisMode     @default(BLIND)
  status           AnalysisStatus   @default(QUEUED)
  verdict          Verdict?
  probability      Float?
  createdAt        DateTime         @default(now())
  finishedAt       DateTime?
  outputs          DetectorOutput[]
  regions          Region[]
  job              AnalysisJob?
  @@index([ownerId, createdAt])
}

model AnalysisJob {
  id          String    @id @default(uuid()) @db.Uuid
  analysisId  String    @unique @db.Uuid
  status      JobStatus @default(QUEUED)
  attempts    Int       @default(0)
  priority    Int       @default(0)
  lockedBy    String?
  lockedAt    DateTime?
  error       String?
  analysis    Analysis  @relation(fields: [analysisId], references: [id], onDelete: Cascade)
  @@index([status, priority, id])
}

enum Verdict        { EDITED NOT_EDITED INCONCLUSIVE }
enum AnalysisMode   { BLIND REFERENCE }
enum AnalysisStatus { QUEUED RUNNING DONE FAILED }
enum JobStatus      { QUEUED RUNNING DONE FAILED }
```

7.3 Claiming a job safely

Several workers may poll at once. This statement lets each job be taken by exactly one worker, and survives crashes because a stale lock (older than the job time limit) is reclaimed.

```
worker claim query

UPDATE analysis_jobs SET status = 'RUNNING', locked_by = $1, locked_at = now(), attempts = attempts + 1
WHERE id = (
  SELECT id FROM analysis_jobs
  WHERE (status = 'QUEUED' OR (status = 'RUNNING' AND locked_at < now() - interval '15 minutes'))
    AND attempts < 3
  ORDER BY priority DESC, id
  FOR UPDATE SKIP LOCKED
  LIMIT 1
)
RETURNING id, analysis_id;
```

7.4 Security and storage rules

- **Row-level security on every table**, with no policies for the public roles: only the API (service role, server-side) reads and writes. The browser never holds the service key.
- **Buckets**: `originals`, `artifacts`, `datasets`, `models`. All private. Paths are prefixed with the owner id: `originals/{owner_id}/{image_id}.{ext}`.
- **Signed URLs** for upload and download, valid for minutes, created by the API after checking ownership.
- **Multi-tenant isolation**: every query that returns user data filters by owner id; add a test that user A cannot read user B's analysis.

- **Retention**: delete originals and artifacts after a configurable period unless the user keeps them; record deletion in audit_events.

MEASUREMENT

8. Evaluation: how coverage and accuracy are measured

The core rule
Every number comes with: the dataset and split, the edit type, the edit size bucket, the post-processing, the metric, the value and a 95% confidence interval. A number without these is not reported.

8.1 Image-level metrics (is this image edited?)

- **ROC-AUC** and **average precision** per edit family and size bucket.
- **True-positive rate at a fixed false-positive rate** (TPR at FPR 1% and 5%): this is what matters to users, because false alarms on genuine photos destroy trust.
- **Balanced accuracy** at the chosen operating thresholds, plus the **inconclusive rate**.

8.2 Pixel-level metrics (where?)

- **F1**, **IoU** and **MCC** between predicted and ground-truth masks, computed per image then averaged; also at the best threshold per image (oracle) only as an upper bound, clearly labelled.
- **Region recall**: share of ground-truth regions with IoU above 0.5 with some predicted region; and the number of false regions per image.

8.3 Calibration

- **Expected calibration error (ECE)** with 15 bins, **Brier score** and a reliability diagram, on the held-out test split.

8.4 Generalisation and robustness

- **Leave-one-tool-out**: train without generative tool X, test on X. Report a table of tools x metrics. This is the honest measure of how the system behaves on the next new AI tool.
- **Robustness matrix**: each metric under each post-processing (JPEG quality, resize, social-media pipeline).
- **Hand-made commercial-tool set**: test only, never trained on.
- **Confidence intervals** by bootstrap over source photos (1,000 resamples).

8.5 Report template

Slice	n	AUC	TPR @ FPR 5%	Pixel F1	IoU	ECE
Generative fill, 2-5%, JPEG q85	...	...	...	...	...	...
Generative fill, unseen tool	...	...	...	...	...	...
Splicing, 5-10%, PNG	...	...	...	...	...	...
Tiny edits, under 1%	...	...	...	...	...	...
Unedited, WhatsApp-like pipeline (FPR only)	...	n/a	FPR = ...	n/a	n/a	...

Fill this table from the evaluation_metrics table, never by hand. The report generator script reads the database and writes Markdown and PDF.

SETUP

9. Repository, setup and configuration

9.1 Folder structure

```
recommended layout (monorepo)

image-forensics/
  apps/
    web/                    React + Vite + Tailwind app
      src/pages/            Upload, AnalysisResult, History, Evaluation
      src/components/       HeatmapOverlay, MaskList, EvidencePanel
      src/lib/api.ts        typed API client (generated from OpenAPI)
    api/                    Node + Express + TypeScript
      src/routes/           uploads.ts, analyses.ts, models.ts, admin.ts
      src/services/         storage.ts, queue.ts, analyses.ts, audit.ts
      src/middleware/       auth.ts, validate.ts, rateLimit.ts, errors.ts
      prisma/               schema.prisma, migrations/
      tests/                Jest + Supertest
  services/
    forensics/              Python 3.11
      app/main.py           FastAPI: /health, /ready, admin endpoints
      app/worker.py         job loop: claim -> run -> write
      app/io/               decode.py (HEIC, EXIF), storage.py, db.py
      app/detectors/        metadata.py, jpeg_double.py, ela.py, noise.py,
                            copymove.py, resampling.py, truFor.py, segformer.py, ...
      app/reference/        register.py, diff.py, crop.py
      app/fusion/           fuse.py, calibrate.py, decide.py, regions.py
      datasets/             generate.py, postprocess.py, splits.py
      training/             train_seg.py, train_fusion.py, configs/
      evaluation/           run_eval.py, metrics.py, report.py
      tests/                pytest, golden images
  packages/shared/          OpenAPI spec, shared TypeScript types
  infra/                    Dockerfiles, render.yaml, GitHub Actions
  docs/                     ARCHITECTURE.md, LICENCES.md, RESULTS.md, RUNBOOK.md
  data/                     (git-ignored) local datasets and model weights
```

9.2 Prerequisites

Tool	Version	Notes
Node.js	20 LTS	With npm or pnpm
Python	3.11	Use a virtual environment (venv or uv)
Docker Desktop	current	For the worker image and a local Postgres when offline
Git + GitHub	current	Protected main branch, pull requests with review
GPU access	one 24 GB GPU for training	Rented by the hour (check current prices before starting)
Supabase project	free or Pro tier	Pick the region closest to your users and your API host

9.3 Setup steps

1. Create the GitHub repository with the folder structure above and a README that explains how to run each part.
2. Create a Supabase project. Copy the **pooler connection string (port 6543)** and the **direct connection string (port 5432)**, the project URL and the service-role key into `apps/api/.env` (never commit it).
3. Create the four private storage buckets. Turn on RLS for the public schema.
4. Run `npx prisma migrate dev` in `apps/api`, then `npm run dev`.

5. Create the Python venv in `services/forensics`, `pip install -r requirements.txt`, run `uvicorn app.main:app --reload` and `python -m app.worker`.
6. Run `npm run dev` in `apps/web`. Upload a test image and watch the job go from queued to done.

9.4 Environment variables

```
.env.example (copy and fill; never commit real values)

# apps/api/.env
DATABASE_URL="postgresql://...pooler.supabase.com:6543/postgres?pgbouncer=true&connection_limit=5"
DIRECT_URL="postgresql://...pooler.supabase.com:5432/postgres"       # migrations use this
SUPABASE_URL="https://<project>.supabase.co"
SUPABASE_SERVICE_ROLE_KEY="..."                                       # server only
JWT_SECRET="<64 random bytes>"
MAX_UPLOAD_BYTES=52428800
CORS_ORIGIN="http://localhost:5173"

# services/forensics/.env
DATABASE_URL="postgresql://...pooler.supabase.com:6543/postgres"
SUPABASE_URL="https://<project>.supabase.co"
SUPABASE_SERVICE_ROLE_KEY="..."
WORKER_ID="worker-local-1"
DEVICE="cpu"            # or cuda
MODEL_MANIFEST="models/manifest.json"
JOB_TIMEOUT_SECONDS=900
```

Prisma with the Supabase pooler
Use the transaction pooler (port 6543) with `pgbouncer=true` for the running app, and the direct or session URL (port 5432) as `directUrl` for migrations. Prepared statements do not work through the transaction pooler without that flag.

CONTRACT

10. API design

The API is described in an OpenAPI 3.1 file in `packages/shared`; the web client types are generated from it. All endpoints are under `/v1`, use JSON, and return errors as `{ "error": { "code", "message" } }`.

Method + path	Purpose	Main responses
POST /v1/uploads	Get a signed upload URL for an image (mime, bytes, sha256 declared)	201 { uploadId, url, expiresAt } / 413 / 415
POST /v1/analyses	Start an analysis: { uploadId, referenceUploadId?, options? }. Header Idempotency-Key.	202 { analysisId, status } / 404 / 409 / 429
GET /v1/analyses/:id	Status and, when done, the full result	200 result / 404
GET /v1/analyses/:id/artifacts	Short-lived signed URLs for the heatmap and each mask	200 { heatmapUrl, masks[] }
GET /v1/analyses	Paged history for the signed-in user	200 { items, nextCursor }
DELETE /v1/analyses/:id	Delete the analysis and its files	204
GET /v1/models	Active detector and model versions with their tags (proven / experimental)	200
POST /v1/admin/evaluations	Start an evaluation run on a dataset (admin only)	202
GET /v1/health, /v1/ready	Liveness, and readiness including database reachability	200 / 503

10.1 Result shape

```
GET /v1/analyses/:id (done)

{
  "analysisId": "9a1c...",
  "status": "DONE",
  "mode": "BLIND",
  "verdict": "EDITED",                      // EDITED | NOT_EDITED | INCONCLUSIVE
  "probability": 0.91,                      // calibrated; see /v1/models for calibration date
  "summary": "Evidence of local editing in 1 region (about 3.2% of the image).",
  "regions": [
    { "id": "r1", "bbox": { "x": 812, "y": 402, "w": 230, "h": 190 }, "areaPct": 3.2,
      "confidence": 0.88, "editTypeGuess": "generative_fill", "maskUrl": "<signed, 10 min>" }
  ],
  "evidence": [
    { "detector": "trufor", "version": "1.0.0", "score": 0.93, "reliability": 0.81, "tag": "PROVEN" },
    { "detector": "noise_residual", "version": "0.3.0", "score": 0.71, "tag": "PROVEN" },
    { "detector": "inpaint_seg", "version": "0.2.0", "score": 0.89, "tag": "EXPERIMENTAL" },
    { "detector": "metadata", "version": "0.1.0", "score": 0.20, "notes": ["No camera model in EXIF"] }
  ],
  "warnings": ["Image was recompressed; small edits may be missed."],
  "limitations": "No evidence found is not proof of authenticity."
}
```

10.2 API rules

- Validate every body, query and header with Zod; reject unknown fields.
- Rate limits per user and per IP; upload size limit enforced on the signed URL and re-checked by the worker.
- Idempotency: the same Idempotency-Key within 24 hours returns the same analysis instead of creating a new one.
- No stack traces or internal paths in error responses; log them server-side with a request id.
- Authentication with short-lived access tokens and a refresh flow; authorisation checks on every resource by owner id.

STEP BY STEP

11. Implementation phases

Ten phases, done in order, all within the one-week deadline. Each phase ends with a short demo and a review against its acceptance criteria. Targets marked "target" are goals to be agreed with your mentor after the first baseline, not promises.

Phase 0: Foundations

Learn	• Read: Verdoliva, "Media Forensics and DeepFakes: an overview" (2020); the TruFor paper; Bianchi and Piva on double JPEG. • JPEG basics: 8x8 DCT, quantisation, chroma subsampling. Noise residuals. What a segmentation mask is. • Prisma + Supabase basics; FastAPI basics; how SKIP LOCKED queues work.
Build	• Monorepo skeleton, linters (ESLint, Prettier, Ruff, mypy), pre-commit hooks, GitHub Actions running all tests. • docs/ARCHITECTURE.md (your own words), docs/LICENCES.md (empty table to fill).
Test	• CI passes on an empty project; one sample unit test per language.
Deliver	• Repository link, a one-page summary of the papers in your own words.
Accept	• CI green on main; a reviewer can clone and run every part with the README in under one hour.
Risks	• Spending too long reading: read only what the next phase needs, then build.

Phase 1: Dataset and ground truth

Learn	• Dataset licences; data leakage; how inpainting models take a mask; COCO segmentation format.
Build	• Download and register public datasets (Section 6.1) with licence notes. • generate.py for every edit family in 6.2, with exact masks, area buckets and post-processing variants. • splits.py: split by source photo, hold out at least one generative tool. • Load everything into datasets / dataset_items and the datasets bucket.
Test	• Mask checks: every mask non-empty; measured area within 0.5 percentage points of the target bucket. • Leakage check: no source photo appears in two splits (automated test). • Visual spot check of 100 random items, recorded in a sheet.
Deliver	• Dataset card (docs/DATASET.md): counts per family, size, tool, post-process; licences.
Accept	• At least the minimum sizes in 6.2 (or an agreed reduced plan); zero leakage; regeneration from seed reproduces identical files.
Risks	• GPU time for inpainting: start generation early and in batches; commercial-tool images are slow to make by hand.

Phase 2: Platform skeleton (end to end with a dummy detector)

Learn	• Signed uploads, RLS, Prisma migrations, Zod, React Query polling.
Build	• All tables in Section 7 with migrations; buckets; signed upload flow. • API endpoints in Section 10 with validation, auth, rate limits, audit log. • Worker loop with the claim query, timeouts, retries and a dummy detector that returns a random map. • Web pages: Upload, Result (heatmap overlay), History.
Test	• Jest + Supertest for every endpoint, including user A cannot read user B (expect 404). • Worker tests: two workers never take the same job; a crashed job is reclaimed after the lock timeout. • Playwright smoke test: upload, wait, see the heatmap.
Deliver	• Demo of the full flow; OpenAPI spec; docs/RUNBOOK.md (how to run, reset, inspect jobs).
Accept	• Upload to result in under 60 seconds locally with the dummy detector; RLS on every table; all tests green.
Risks	• Connection limits on the pooler: keep the API pool small (5) and reuse one Prisma client.

Phase 3: Classical detectors

Learn	• DCT histograms, double quantisation, residual filters (SRM kernels), keypoint matching, resampling spectra.
Build	• Detectors as plug-ins with one interface: run(image) -> { score, map, reliability?, details }. • metadata, jpeg_double, ela (weak signal), noise_residual, copymove (SIFT + g2NN + RANSAC), resampling, jpeg_grid (crop hint). • Each writes its map to storage and a detector_outputs row with its version and timing.
Test	• Golden-image tests: known edited and unedited images with expected outputs within tolerances. • Per-detector evaluation on the validation split: AUC per family; timing on a 12-megapixel image.
Deliver	• Results table per detector and family; notes on where each fails.
Accept	• Every detector runs on a 12 MP image in under 10 s on CPU (target), never crashes on any test image (including HEIC, PNG, CMYK JPEG, 16-bit PNG), and has measured AUC per family.
Risks	• ELA misread as proof: keep it labelled as a weak signal in the UI.

Phase 4: Reference comparison mode

Learn	• Homographies, RANSAC, SSIM, LPIPS, exposure and colour matching.
Build	• register.py (SIFT, ratio test, RANSAC; quality gate on inliers and reprojection error). • diff.py (aligned tile differences, SSIM map, noise-floor threshold), crop.py (crop box), global change report. • API and UI support for uploading the original.
Test	• Synthetic tests: known crop, rotate, scale, filter and local edit; masks compared with ground truth.
Deliver	• Reference-mode demo and metrics.
Accept	• Target: pixel F1 at least 0.85 on local edits of 1% area or more after crop and resize; correct crop box within 2 px; refuses to compare when alignment fails instead of guessing.
Risks	• Repetitive textures confuse registration: use the inlier gate and report "cannot align".

Phase 5: Pretrained deep models

Learn	• How to load research checkpoints, input normalisation, sliding-window inference on large images, half precision.
Build	• Wrappers for at least two of: TruFor, CAT-Net, IML-ViT, MVSS-Net (licences recorded first). • Model registry entries with SHA-256; tiling for images larger than the model input; GPU and CPU paths.
Test	• Sanity check: reproduce a published number on a public benchmark subset within about 5 points. • Speed and memory on a 12 MP image, CPU and GPU.
Deliver	• Baseline evaluation table: each model on each family, size and post-process.
Accept	• Models run end to end in the worker; baselines recorded in evaluation_metrics; any licence blocker documented.
Risks	• Licence forbids the intended use: keep the model for research comparison only and rely on your own fine-tuned model.

Phase 6: Fine-tuning and generative edits

Learn	• Segmentation losses (BCE + Dice, focal), class imbalance for tiny regions, mixed precision, early stopping, augmentation that preserves forensic traces.
Build	• train_seg.py: SegFormer or UNet (segmentation-models-pytorch) trained on your dataset; inputs RGB plus a noise-residual channel. • Tiny-edit emphasis: oversample small regions, loss weighting by region size. • Experimental image-level AI detectors (UnivFD-style CLIP probe, NPR-style features) as extra fusion inputs. • Every run tracked in MLflow with git SHA, data version, hyperparameters.
Test	• Validation every epoch; final numbers only on the test split, once. • Leave-one-tool-out experiments for each generative tool.
Deliver	• Trained model in the registry, training report with curves and failure examples.
Accept	• Target: pixel F1 on held-out generative fill improves by at least 10 points over the best pretrained baseline, or a documented negative result with analysis.
Risks	• Overfitting to one generator: mix tools and post-processing; watch the unseen-tool gap.

Phase 7: Fusion, calibration, false-positive control

Learn	• Stacking, logistic regression, gradient boosting, temperature scaling, isotonic regression, ECE.
Build	• fuse.py trained on the validation split; calibrate.py; decide.py with two thresholds and the inconclusive band; regions.py. • Hard-negative mining: add the unedited images most often flagged back into training of the fusion step. • Abstain rules: too small, too compressed, detectors disagree, alignment failed.
Test	• Operating point chosen on validation for a target false-positive rate; checked once on test.
Deliver	• Calibration report with reliability diagrams; chosen thresholds with reasons.
Accept	• Targets: FPR at most 3% on unedited, normally processed photos at the "edited" threshold; ECE at most 0.05 on test; inconclusive rate reported per slice.
Risks	• Thresholds tuned on test by accident: the evaluation code must refuse to tune on the test split.

Phase 8: Full evaluation and unseen tools

Learn	• Bootstrap confidence intervals; writing a fair results section.
Build	• run_eval.py over every slice; report.py producing docs/RESULTS.md and a PDF from the database.
Test	• Hand-made commercial-tool set; robustness matrix; leave-one-tool-out table.
Deliver	• Results report with limitations: which edits are missed, how often, under which conditions.
Accept	• Every number in the report traceable to an evaluation_runs row; confidence intervals on all headline numbers.
Risks	• Pressure to report only good numbers: the report must include the worst slices.

Phase 9: UI polish, deployment, handover

Learn	• Docker multi-stage builds, Render services, Vercel envs, production migrations.
Build	• Result page: verdict wording rules, region list, heatmap opacity, evidence with PROVEN / EXPERIMENTAL tags, downloadable report. • Dockerfiles; Render web service (API) and background worker; Vercel for the web app; prisma migrate deploy in the release step. • Monitoring: structured logs with request and job ids; alert on job failure rate.
Test	• Staging run on the test split through the deployed system; load test of 20 concurrent analyses.
Deliver	• Live URLs, runbook, architecture document, results report, recorded demo, handover session.
Accept	• p95 time to result for a 12 MP image under 60 s on the CPU path (target); no secrets in the repository; a new developer can deploy from the runbook.
Risks	• Deep models too slow on CPU: route to a GPU host or run only the classical set on CPU, and say so in the result.

QUALITY

12. Testing strategy

Level	Tool	What is tested	Gate
Unit (Python)	pytest	Each detector, fusion, calibration, region extraction, metrics	Every PR
Unit (TS)	Jest	Services, validation schemas, storage path rules	Every PR
API integration	Supertest + test database	Every endpoint, auth, ownership, idempotency, error shapes	Every PR
Golden images	pytest	Fixed inputs with expected outputs within tolerance; catches silent changes	Every PR
Benchmark regression	run_eval.py on a small frozen subset	AUC and pixel F1 must not drop more than 2 points from main	PRs touching detectors or models
End to end	Playwright	Upload, analyse, view result, delete	Before deploy
Security	npm audit, pip-audit, a cross-tenant test suite, secret scanning	Dependencies, isolation, leaked keys	Before deploy
Load	k6 or Locust	20 concurrent analyses; queue drains; no memory growth	Before launch

12.1 Inputs every test suite must include

- JPEG (baseline and progressive), PNG (8 and 16 bit), WebP, HEIC phone photos, CMYK JPEG, images with EXIF rotation, a 1x1 image, a 100-megapixel image, a truncated file, a file renamed to the wrong extension, and an image with no metadata.

PRODUCTION

13. Deployment

1. **Database**: production Supabase project in the chosen region; automated backups on; migrations run with `prisma migrate deploy` in the API release step, never by hand.
2. **API**: Docker image on a Render web service in the same region; health check on /v1/ready; at least 2 instances when traffic grows.
3. **Worker**: Render background worker (CPU) for classical detectors. For deep models either a GPU host (any provider with on-demand GPUs) running the same image with DEVICE=cuda, or CPU with the slower path documented.
4. **Web**: Vercel project with the API URL as an environment variable; preview deployments per pull request.
5. **Secrets**: only in the host's environment settings; rotate on any leak; never in the repository or the browser.
6. **Release checklist**: CI green, benchmark gate green, migrations reviewed, model checksums match the registry, rollback plan written.
7. **After deploy**: run the staging evaluation subset against production and compare with the last report.

> **Restarts and long jobs**
> Every deploy restarts the services. Make jobs safe to retry (the claim query reclaims stale jobs) and give the worker a graceful shutdown that finishes or releases the current job.

HONESTY

14. Risks and limitations

Risk / limitation	Effect	Mitigation
New AI editing tools appear constantly	Generative detectors lose accuracy on unseen tools	Leave-one-tool-out reporting; regular retraining; reference mode where possible
Heavy recompression and resizing	Destroys compression and noise traces; tiny edits disappear	Robustness training; inconclusive verdict; warnings in results
Tiny edits (under 1%)	Often below the noise floor in blind mode	Report miss rate honestly; reference mode for these cases
False positives on genuine photos	Loss of trust; harm to the person accused	Fixed-FPR operating point; hard negatives; never say "fake", show evidence
Adversarial evasion	A determined attacker can hide edits	State it plainly; ensemble of diverse detectors; do not publish thresholds
Model and dataset licences	Some models cannot be used commercially	Licence register; your own fine-tuned model as the production path
Small evaluation sets	Wide uncertainty	Bootstrap intervals; grow the test set over time
Compute cost	GPU time for training and inference	Budget agreed up front; CPU fallback; batch generation
Privacy	User photos may be personal	Private buckets, retention limits, deletion endpoint, no training on user uploads without consent

14.1 Proven versus experimental at a glance

Proven (within known limits)	Experimental (research)
• Metadata and C2PA reading • Double JPEG and quantisation analysis • Noise residual inconsistency, Noiseprint • Copy-move with keypoints • Resampling detection • Reference registration and difference maps • TruFor-class localisation on known manipulation types • Calibration with temperature scaling	• Detecting AI inpainting from unseen tools • Fully AI-generated image detection in the wild • AI upscaling and enhancement detection • Tiny-edit detection without the original • Telling camera processing from global filters

READING LIST

15. Technical references

Read the abstract, method and limitations of each before using it. Check each paper's code licence before use.

1. L. Verdoliva, "Media Forensics and DeepFakes: an Overview", IEEE JSTSP, 2020.
2. F. Guillaro, D. Cozzolino, A. Sud, N. Dufour, L. Verdoliva, "TruFor: Leveraging all-round clues for trustworthy image forgery detection and localization", CVPR 2023.
3. M.-J. Kwon, S.-H. Nam, I.-J. Yu, H.-K. Lee, C. Kim, "Learning JPEG compression artifacts for image manipulation detection and localization" (CAT-Net), IJCV 2022.
4. D. Cozzolino, L. Verdoliva, "Noiseprint: a CNN-based camera model fingerprint", IEEE TIFS, 2020.
5. D. Cozzolino, G. Poggi, L. Verdoliva, "Splicebuster: a new blind image splicing detector", WIFS 2015.
6. T. Bianchi, A. Piva, "Image forgery localization via block-grained analysis of JPEG artifacts", IEEE TIFS, 2012.
7. A. C. Popescu, H. Farid, "Exposing digital forgeries by detecting traces of resampling", IEEE TSP, 2005.

8. M. Kirchner, "Fast and reliable resampling detection by spectral analysis of fixed linear predictor residue", ACM MM&Sec 2008.
9. I. Amerini et al., "A SIFT-based forensic method for copy-move attack detection and transformation recovery", IEEE TIFS, 2011.
10. J. Lukas, J. Fridrich, M. Goljan, "Digital camera identification from sensor pattern noise", IEEE TIFS, 2006.
11. Y. Wu, W. AbdAlmageed, P. Natarajan, "ManTra-Net", CVPR 2019; X. Chen et al., "MVSS-Net", ICCV 2021; X. Ma et al., "IML-ViT", 2023.
12. U. Ojha, Y. Li, Y. J. Lee, "Towards universal fake image detectors that generalize across generative models" (UnivFD), CVPR 2023.
13. C. Tan et al., "Rethinking the up-sampling operations in CNN-based generative network for generalizable deepfake detection" (NPR), CVPR 2024.
14. Z. Wang et al., "DIRE for diffusion-generated image detection", ICCV 2023.
15. R. Suvorov et al., "Resolution-robust large mask inpainting with Fourier convolutions" (LaMa), WACV 2022.
16. Datasets: CASIA (Dong et al., 2013); Columbia (Hsu and Chang, 2006); COVERAGE (Wen et al., 2016); IMD2020 (Novozamsky et al., 2020); DEFACTO (Mahfoudi et al., 2019); TGIF (Mareen et al., 2024); GenImage (Zhu et al., 2023); RAISE (Dang-Nguyen et al., 2015).
17. C. Guo, G. Pleiss, Y. Sun, K. Q. Weinberger, "On calibration of modern neural networks", ICML 2017 (temperature scaling).
18. C2PA technical specification (c2pa.org); Supabase documentation (database, storage, RLS, connection pooling); Prisma documentation.

HANDOVER

16. Final acceptance checklist

#	Item	Evidence
1	Blind and reference modes work end to end in production	Live URLs, demo video
2	Verdict is three-way and wording never claims authenticity or 100%	UI screenshots, API samples
3	Pixel masks and per-region confidence for every edited result	API sample, stored masks
4	Calibrated probability with ECE reported on the test split	Calibration report
5	FPR on unedited, normally processed photos measured and within the agreed target	RESULTS.md slice table
6	Leave-one-tool-out and robustness tables with confidence intervals	RESULTS.md
7	Every technique labelled proven or experimental in docs and UI	Docs, UI
8	Licence register complete; no blocked component in the production path	LICENCES.md
9	RLS on every table; cross-tenant tests pass; no secrets in git	CI logs, secret scan
10	Setup from README in under one hour by someone new	Reviewer sign-off
11	Limitations section lists missed edit types and their measured miss rates	RESULTS.md

A last word

Build a system that catches edits other tools miss, and prove it with numbers. When it misses something, find out why and make it better. Show the evidence behind every answer.